Лабораторная работа №12

Тема работы: «Разработка рекурсивных алгоритмов и программ»

1 Цель работы

Сформировать умение разрабатывать и производить отладку алгоритмов и программ с использованием рекурсивных функций

2 Задание

1. С помощью признака делимости на 3 проверить, кратно ли данное натуральное число 3. Написать рекурсивную функцию (Операцию % 3 не используйте).

12. Разработать рекурсивную функцию нахождения наибольшего общего делителя двух чисел m и n используя алгоритм Евклида, в соответствии с которым НОД(a,b)= НОД(max(a,b)-min(a,b),min(a,b)).

14. Разработать рекурсивную функцию нахождения значения функции Аккермана, которая определяется для всех неотрицательных целых аргументов m и n следующим образом: A(0,n)=n+1; A(m,0)=A(m-1,1), если (m>0); A(m,n)=A(m-1,A(m,n-1)), если (m, n >0).

3 Оснащение

ПК, задание по варианту, среда разработки Visual Studio.

4 Порядок выполнения работы

Код программы:

```
#include <iostream>
#include <string>
using namespace std;

int sumDigits(const string& num, int index = 0) {
    if (index == num.size()) return 0;
    return (num[index] - '0') + sumDigits(num, index + 1);
}

int isDivisibleBy3(const string& num) {
    int sum = sumDigits(num);

    if (sum > 9) {
        return isDivisibleBy3(to_string(sum));
    }

    if (sum == 0 || sum == 3 || sum == 6 || sum == 9) {
        return 1;
    }
    else {
        return 0;
    }
}
```

```
24.
25. int NOD(int m, int n) {
26.     if (m == n) return m;
27.     if (m > n) return NOD(m - n, n);
28.     return NOD(m, n - m);
29. }
30. int ackermann(int m, int n) {
31.     if (m == 0) return n + 1;
32.     else if (n == 0) return ackermann(m - 1, 1);
33.     else return ackermann(m - 1, ackermann(m, n - 1));
34. }
35.
36.
37. int main() {
38.     setlocale(LC_ALL, "RU");
39.
40.     while (true) {
41.         cout << "Выберите задание:\n";
42.         cout << "1 - Проверка кратности числа 3\n";
43.         cout << "2 - НОД двух чисел\n";
44.         cout << "3 - Функция Аккермана\n";
45.         cout << "0 - Выход\n";
46.
47.         int choice;
48.         cin >> choice;
49.
50.         if (choice == 0) break;
51.
52.         if (choice == 1) {
53.             string n;
54.             cout << "Введите число: ";
55.             cin >> n;
56.
57.             if (isDivisibleBy3(n)) {
58.                 cout << "Число кратно 3\n";
59.             }
60.             else {
61.                 cout << "Число не кратно 3\n";
62.             }
63.         }
64.         else if (choice == 2) {
65.             int m, n;
66.             cout << "Введите два числа: ";
67.             cin >> m >> n;
68.
69.             cout << "НОД(" << m << ", " << n << ") = " << NOD(m, n) << endl;
70.         }
71.         else if (choice == 3) {
72.             int m, n;
73.             cout << "Введите m и n: ";
74.             cin >> m >> n;
75.
76.             if (m > 3 || n > 10) {
77.                 cout << "Слишком большие значения! Возможен выход за пределы стека." << endl;
78.             }
79.             else {
80.                 cout << "A(" << m << ", " << n << ") = " << ackermann(m, n) << endl;
81.             }
82.         }
83.         else {
84.             cout << "Неверный выбор задания!" << endl;
85.         }
86.     }
87.
88.     return 0;
89. }
90.
```

Результат работы программы:

```
Выберите задание:
1 - Проверка кратности числа 3
2 - НОД двух чисел
3 - Функция Аккермана
0 - Выход
1
Введите число: 9
Число кратно 3
Выберите задание:
1 - Проверка кратности числа 3
2 - НОД двух чисел
3 - Функция Аккермана
0 - Выход
1
Введите число: 13
Число не кратно 3
```

Рисунок 1 - Результат выполнения задачи №1

```
Выберите задание:
1 - Проверка кратности числа 3
2 - НОД двух чисел
3 - Функция Аккермана
0 - Выход
3
Введите m и n: 2 0
A(2, 0) = 3
Выберите задание:
1 - Проверка кратности числа 3
2 - НОД двух чисел
3 - Функция Аккермана
0 - Выход
3
Введите m и n: 0 2
A(0, 2) = 3
Выберите задание:
1 - Проверка кратности числа 3
2 - НОД двух чисел
3 - Функция Аккермана
0 - Выход
3
Введите m и n: 1 2
A(1, 2) = 4
Выберите задание:
1 - Проверка кратности числа 3
2 - НОД двух чисел
3 - Функция Аккермана
0 - Выход
```

Рисунок 2 - Результат выполнения задачи №14

```
Выберите задание:
1 - Проверка кратности числа 3
2 - НОД двух чисел
3 - Функция Аккермана
0 - Выход
2
Введите два числа: 120
26
НОД(120, 26) = 2
```

Рисунок 3 - Результат выполнения задачи №12

Контрольные вопросы:

1. Дайте краткое понятие рекурсии в контексте программирования.

Рекурсия — это метод, при котором функция вызывает саму себя для решения задачи через разбиение её на более простые подзадачи.

2. Раскройте значение применения рекурсии при разработке алгоритмов.

Рекурсия упрощает разработку алгоритмов, позволяя выразить сложные задачи через повторяющиеся структуры, делает код более понятным и компактным.

3. Сформулируйте сущность базового случая в рекурсивных алгоритмах и его значение.

Базовый случай — это условие завершения рекурсии, предотвращающее бесконечные вызовы и обеспечивающее корректное завершение алгоритма.

4. Изложите особенности работы стека вызовов при выполнении рекурсивных функций.

Каждый вызов рекурсивной функции сохраняется в стеке вызовов; при достижении базового случая стек начинает разворачиваться, возвращая результаты в обратном порядке.

5. Раскройте особенности использования рекурсивных алгоритмов для решения задач, например, вычисление факториала или поиск элемента в списке.

Рекурсивные алгоритмы удобны для задач с естественной иерархией (факториал, обход дерева, поиск в списке), но требуют контроля базового случая и могут быть менее эффективны по памяти из-за глубины стека.